

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
SPRING 2026**



AgriHome project logo: leaf and camera lens motif for vision-based agricultural monitoring.

AGRIHOME

**GOVINDA AWASTHI
LAXMAN BHUSHAL
THUC DANG
SAGAR KHADKA
MUHAMMAD SAAD MEMON
ALEX TABLIZO
NATHAN NGUYEN TRAN**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	03.04.2026	Team	Document created; initial architecture draft
1.0	03.04.2026	Team	Layered overview, figures, PDF verified
1.1	03.04.2026	Team	Raised system-level abstraction in overview sections
2.0	03.04.2026	GA, LB, TD, SK, MSM, AT, NNT	Restructure to CSE 4316 ADS format; subsystems, interfaces; team roster
3.0	07.23.2026	Team	Summer update: Pi/Moonraker edge layer, Devices UI, capture APIs
3.1	07.28.2026	Team	Klipper edge: fswebcam primary capture; klipper_url rename

CONTENTS

1	Introduction	9
1.1	Purpose	9
1.2	Scope	9
1.3	System summary	9
1.4	Document organization	10
2	Project Overview	11
2.1	Presentation layer description	11
2.2	Application layer description	12
2.3	Processing layer description	12
2.4	Data layer description	12
2.5	Hardware layer description	13
2.6	Layered architecture figure	13
2.7	System workflow overview	14
3	System Description	15
3.1	Presentation layer composition	15
3.2	Application layer composition	15
3.3	Processing layer composition	16
3.4	Data layer composition	16
3.5	Hardware layer composition	16
3.6	End-to-end data flow summary	16
4	Data Layer Subsystems	17
4.1	Write coordination subsystem	17
4.1.1	Assumptions	17
4.1.2	Responsibilities	17
4.1.3	Subsystem interfaces	17
4.2	Integrity and constraint enforcement subsystem	17
4.2.1	Assumptions	18
4.2.2	Responsibilities	18
4.2.3	Subsystem interfaces	18
4.3	Query and transaction execution subsystem	18
4.3.1	Assumptions	18
4.3.2	Responsibilities	18
4.3.3	Subsystem interfaces	19
4.4	PostgreSQL domain database subsystem	19
4.4.1	Assumptions	19
4.4.2	Responsibilities	19
4.4.3	Subsystem interfaces	19
4.5	Edge domain persistence subsystem	20
4.5.1	Assumptions	20
4.5.2	Responsibilities	20
4.5.3	Subsystem interfaces	20
4.6	Image and media storage subsystem	20
4.6.1	Assumptions	20

4.6.2	Responsibilities	21
4.6.3	Subsystem interfaces	21
5	Presentation Layer Subsystems	22
5.1	Authentication and session UI subsystem	22
5.1.1	Assumptions	22
5.1.2	Responsibilities	22
5.1.3	Subsystem interfaces	22
5.2	Dashboard and overview subsystem	22
5.2.1	Assumptions	22
5.2.2	Responsibilities	23
5.2.3	Subsystem interfaces	23
5.3	Tray monitoring console subsystem	23
5.3.1	Assumptions	23
5.3.2	Responsibilities	23
5.3.3	Subsystem interfaces	23
5.4	Plant detail and health history subsystem	24
5.4.1	Assumptions	24
5.4.2	Responsibilities	24
5.4.3	Subsystem interfaces	24
5.5	Mesh and topology subsystem	24
5.5.1	Assumptions	24
5.5.2	Responsibilities	24
5.5.3	Subsystem interfaces	25
5.6	Schedule management subsystem	25
5.6.1	Assumptions	25
5.6.2	Responsibilities	25
5.6.3	Subsystem interfaces	25
5.7	Devices UI subsystem	25
5.7.1	Assumptions	25
5.7.2	Responsibilities	26
5.7.3	Subsystem interfaces	26
5.8	Tray Raspberry Pi panel and Take Picture subsystem	26
5.8.1	Assumptions	26
5.8.2	Responsibilities	26
5.8.3	Subsystem interfaces	26
6	Application Layer Subsystems	27
6.1	API entry and routing subsystem	27
6.1.1	Assumptions	27
6.1.2	Responsibilities	27
6.1.3	Subsystem interfaces	27
6.2	Session and authentication broker subsystem	27
6.2.1	Assumptions	28
6.2.2	Responsibilities	28
6.2.3	Subsystem interfaces	28
6.3	Input validation subsystem	28
6.3.1	Assumptions	28

6.3.2	Responsibilities	28
6.3.3	Subsystem interfaces	29
6.4	Domain orchestration subsystem	29
6.4.1	Assumptions	29
6.4.2	Responsibilities	29
6.4.3	Subsystem interfaces	29
6.5	GraphQL query and mutation facade subsystem	29
6.5.1	Assumptions	30
6.5.2	Responsibilities	30
6.5.3	Subsystem interfaces	30
6.6	Response and error handling subsystem	30
6.6.1	Assumptions	30
6.6.2	Responsibilities	30
6.6.3	Subsystem interfaces	30
6.7	Raspberry Pi edge API subsystem	31
6.7.1	Assumptions	31
6.7.2	Responsibilities	31
6.7.3	Subsystem interfaces	31
6.8	Operator devices API subsystem	31
6.8.1	Assumptions	32
6.8.2	Responsibilities	32
6.8.3	Subsystem interfaces	32
7	Processing Layer Subsystems	33
7.1	Species and leaf inference adapter subsystem	33
7.1.1	Assumptions	33
7.1.2	Responsibilities	33
7.1.3	Subsystem interfaces	33
7.2	Tray vision and layout adapter subsystem	33
7.2.1	Assumptions	34
7.2.2	Responsibilities	34
7.2.3	Subsystem interfaces	34
7.3	Inference normalization subsystem	34
7.3.1	Assumptions	34
7.3.2	Responsibilities	34
7.3.3	Subsystem interfaces	34
7.4	Edge disease hook subsystem	35
7.4.1	Assumptions	35
7.4.2	Responsibilities	35
7.4.3	Subsystem interfaces	35
7.5	Capture schedule runner subsystem	35
7.5.1	Assumptions	35
7.5.2	Responsibilities	36
7.5.3	Subsystem interfaces	36
8	Hardware Layer Subsystems	37
8.1	Raspberry Pi edge host subsystem	37
8.1.1	Assumptions	37

8.1.2	Responsibilities	37
8.1.3	Subsystem interfaces	37
8.2	Klipper / fswebcam capture subsystem	37
8.2.1	Assumptions	38
8.2.2	Responsibilities	38
8.2.3	Subsystem interfaces	38
8.3	agrihome_agent bridge subsystem	38
8.3.1	Assumptions	38
8.3.2	Responsibilities	39
8.3.3	Subsystem interfaces	39
References		40

LIST OF FIGURES

1	AgriHome Vision Console: five-layer architecture with Pi/Klipper edge trust boundaries.	13
2	Subsystem definitions and primary data flow (edge hardware talks to the application layer; application also reads/writes the data layer directly for most CRUD paths).	15
3	Write coordination subsystem: accepts write intents and forwards them toward integrity checks.	17
4	Integrity subsystem: validates operations before execution.	17
5	Query executor: issues SQL and coordinates binary I/O with storage.	18
6	PostgreSQL domain database subsystem (includes edge device and pose tables).	19
7	Edge domain persistence: relational edge tables plus capture binaries.	20
8	Image and media storage subsystem and relationship to application reads.	20
9	Authentication UI: user actions become authenticated API calls.	22
10	Dashboard subsystem requests summary aggregates from the application tier.	22
11	Tray console: drives reads and optional vision requests through the application layer.	23
12	Plant detail subsystem: CRUD and reporting views.	24
13	Mesh subsystem interacts with mesh and member tray aggregates.	24
14	Schedule subsystem persists policies via application APIs.	25
15	Devices UI: operator fleet view over edge device APIs.	25
16	Tray Pi panel: Take Picture and Klipper URL management via devices API.	26
17	API entry: first server hop after the presentation layer.	27
18	Session broker: establishes trusted account context for downstream work.	28
19	Validation subsystem: rejects bad input early.	28
20	Domain orchestration: central business logic and integration point.	29
21	GraphQL facade: thin mapping to domain services.	29
22	Response handler: maps internal results to HTTP/GraphQL responses.	30
23	Raspberry Pi edge API: device-key routes into domain persistence.	31
24	Operator devices API: Take Picture and device administration.	31
25	Species inference adapter: hosted model runtime may be external.	33
26	Tray vision adapter: round-trip inference for tray imagery.	33
27	Normalization: single canonical shape for downstream persistence.	34
28	Edge disease hook: optional classification after Pi capture.	35
29	Capture schedule runner: enqueues edge capture work for agents.	35
30	Pi edge host: register and heartbeat against AgriHome.	37
31	Primary fswebcam capture on the agent; optional server-side HTTP streamer still.	38
32	Edge agent bridges Klipper/fswebcam local capture to AgriHome APIs.	38

LIST OF TABLES

2	Write coordination subsystem interfaces	17
3	Integrity and constraint enforcement subsystem interfaces	18
4	Query and transaction execution subsystem interfaces	19
5	PostgreSQL domain database subsystem interfaces	19
6	Edge domain persistence subsystem interfaces	20
7	Image and media storage subsystem interfaces	21
8	Authentication and session UI subsystem interfaces	22
9	Dashboard and overview subsystem interfaces	23
10	Tray monitoring console subsystem interfaces	23
11	Plant detail and health history subsystem interfaces	24

12	Mesh and topology subsystem interfaces	25
13	Schedule management subsystem interfaces	25
14	Devices UI subsystem interfaces	26
15	Tray Raspberry Pi panel and Take Picture subsystem interfaces	26
16	API entry and routing subsystem interfaces	27
17	Session and authentication broker subsystem interfaces	28
18	Input validation subsystem interfaces	29
19	Domain orchestration subsystem interfaces	29
20	GraphQL facade subsystem interfaces	30
21	Response and error handling subsystem interfaces	30
22	Raspberry Pi edge API subsystem interfaces	31
23	Operator devices API subsystem interfaces	32
24	Species and leaf inference adapter interfaces	33
25	Tray vision and layout adapter interfaces	34
26	Inference normalization subsystem interfaces	34
27	Edge disease hook subsystem interfaces	35
28	Capture schedule runner subsystem interfaces	36
29	Raspberry Pi edge host subsystem interfaces	37
30	Klipper / fswebcam capture subsystem interfaces	38
31	agrihome_agent bridge subsystem interfaces	39

1 INTRODUCTION

1.1 PURPOSE

The **AgriHome Vision Console** is a web-based monitoring and decision-support product for controlled-environment agriculture and laboratory plant studies. Operators and researchers use it to observe tray-level imagery, track individual plants, review automated health assessments, organize trays into logical **meshes**, manage image-capture schedules, and operate **Raspberry Pi / Klipper** edge capture devices. This **Architectural Design Specification (ADS)** records how the product is decomposed into layers and subsystems, how data moves among them, and which interfaces connect internal components to each other and to external partners.

This document is a companion to the team's **System Requirements Specification (SRS)** for the same project.[1] The SRS states *what* the system must do for customers (for example, plant health presentation, automatic capture, environmental data, dashboards, authentication, and secure storage). This ADS states *how* those responsibilities are allocated to software structure: presentation, application orchestration, processing (vision adapters, edge disease hook, schedule runner), persistent data, and hardware edge capture.

1.2 SCOPE

In scope for this ADS are: the logical five-layer architecture (presentation, application, processing, data, hardware); subsystem names, assumptions, responsibilities, and interface summaries; major data flows from user or edge-agent action to storage; identification of optional processing and search backends; Raspberry Pi / Klipper edge integration at the architectural level; and deployment context at the level of application, database, object storage, and bench devices.[8]

Out of scope are: user-interface visual design mockups beyond structural responsibilities; exhaustive REST field specifications (see engineering docs); source file paths and build commands; formal threat models (see edge security notes); and training procedures for ML models.

1.3 SYSTEM SUMMARY

AgriHome addresses limitations of ad hoc spreadsheets, disconnected camera feeds, and manual record-keeping by providing a **single console** that ties together:

- **Identity and access** — authenticated users interact only with their permitted trays, plants, and meshes; edge devices authenticate with hashed device API keys.
- **Observation** — latest captures, charts, and monitoring history support plant status display, historical data, and dashboard-style summaries.
- **Edge capture** — Raspberry Pi devices with Agri-Home/klipper (fswebcam via `save_image.sh`) auto-provision, heartbeat, and deliver imagery via Take Picture (primary agent capture; optional HTTP streamer still) and scheduled multi-pose capture.[6]
- **Interpretation** — health reports and optional model-backed inference (including post-ingest disease analysis) enrich raw imagery where configured.
- **Planning** — schedules express when captures should occur, including destination `raspberry-pi-edge`.
- **Persistence** — relational storage holds authoritative domain and edge state; object storage holds images referenced from that state.

The reference implementation uses a modern full-stack web platform (browser and installable PWA client, server-hosted application, PostgreSQL, file storage, Firebase Authentication, optional HTTP-based inference services, and Python `agrihome_agent` on the Pi). The architecture in this document is described so that those technologies could be substituted behind the same layer boundaries where a future maintainer requires it.

1.4 DOCUMENT ORGANIZATION

Section 2 presents the **project overview** and describes the five primary layers. Section 3 gives a **system description** with a consolidated subsystem-and-data-flow view. Sections 4–8 document **data**, **presentation**, **application**, **processing**, and **hardware** layer subsystems respectively, each with assumptions, responsibilities, and interface tables. The **References** section lists external citations and the companion SRS.

2 PROJECT OVERVIEW

The AgriHome Vision Console is designed using a **layered architecture**. Each layer has a distinct role; layers interact through well-defined interfaces (HTTP APIs between client and server, device-key APIs to edge agents, internal service calls within the server, SQL and file I/O toward stores, optional outbound calls to inference services, and optional HTTP still pulls to LAN streamers). This separation improves modularity, testability, and the ability to scale or replace one concern (for example, swapping a vision provider or webcam path) without rewriting unrelated areas.

The system comprises **five primary layers**:

1. **Presentation Layer** — user-facing experience in the browser or PWA: landing and marketing entry, authentication, dashboard, tray and plant views, mesh management, schedules, **Devices** navigation, tray Raspberry Pi panel with Take Picture, charts, and uploads.
2. **Application Layer** — server-side control: HTTP and GraphQL entry points (including `/api/raspberry-pi/*` and `/api/devices/*`), session and device-key verification, validation, routing of work to domain services, coordination of reads and writes to the data layer, invocation of the processing layer when needed, and optional HTTP streamer still fetches.
3. **Processing Layer** — computer-vision and analytics *adapters*, plus edge post-ingest hooks and the capture schedule runner: optional services that accept images or features and return structured labels, detections, or scores; schedule runners that enqueue `capture_now` work for `raspberry-pi-edge` destinations.
4. **Data Layer** — durable storage: relational database for domain and edge entities (`edge_devices`, `edge_device_commands`, `capture_pose_sequences`, `capture_poses`, `capture_schedules`, `camera_captures`, `plants`, `prediction_results`, `trays`, `meshes`, `reports`), and object or file storage for imagery; optional vector index for similarity search when enabled.
5. **Hardware Layer** — Raspberry Pi hosts running Agri-Home/klipper (`fswebcam / camera-macros/save_image.sh`), optional LAN HTTP streamer bases, and the `agrihome_agent` bridge that registers, heartbeats, captures, and ingests.

2.1 PRESENTATION LAYER DESCRIPTION

The Presentation Layer is the primary interface between **people** and the product. It implements responsive layouts for desktop and mobile, supports installable PWA behavior for field-adjacent use, and communicates with the Application Layer exclusively through HTTPS (including authenticated cookie-based sessions after sign-in).

Users register or sign in, navigate trays and plants, upload photos, trigger or view vision-assisted results, inspect monitoring history, configure schedules, manage meshes, and operate **edge devices** (list devices, edit Klipper/streamer URL, Take Picture from the tray Raspberry Pi panel). The layer is responsible for presenting SRS-mandated capabilities such as health status, historical trends, dashboard summaries, and settings-oriented flows at a user-experience level; it does not embed direct database access.

Key responsibilities:

- Capture user input (forms, file selection, navigation, Take Picture).
- Render charts, tables, imagery, device status, and structured errors clearly.
- Invoke application APIs and display responses or structured errors.

- Encapsulate client-only concerns (progressive enhancement, local caching of shell assets where applicable).

2.2 APPLICATION LAYER DESCRIPTION

The Application Layer is the **central coordinator**. It receives every authenticated request from the Presentation Layer, interprets it, enforces authorization tied to the authenticated identity, validates payloads, delegates to domain-oriented modules (tray and plant lifecycle, mesh topology, schedules, monitoring logs, file references, edge device actions), and returns responses.

It also terminates **device-key** traffic from Raspberry Pi agents (`/api/raspberry-pi/register`, `heartbeat`, `ingest`, `commands`, `poses`, `capture-plan`) and operator device actions (`/api/devices`). For Take Picture, this layer primarily queues `capture_now` for the agent (`fswebcam`); when `klipper_url` is set it may optionally fetch an HTTP streamer still first. It bridges to the Processing Layer for species/disease classification and schedule-driven capture enqueue.

Key responsibilities:

- Expose a stable API surface for the console, edge agents, and future integrators.
- Maintain session integrity, ownership rules, and hashed device API keys.
- Orchestrate transactions across relational updates and file metadata (including multipart ingest).
- Isolate failures in optional backends so core monitoring remains usable when possible.

2.3 PROCESSING LAYER DESCRIPTION

The Processing Layer performs **compute-intensive or model-based** work and **asynchronous capture orchestration** that should not block the interactive UI thread. In AgriHome this includes, when configured, **species or leaf-level classification**, **tray-level vision**, the **edge disease hook** after Pi ingest, and the **capture schedule runner** that enqueues edge capture commands for destinations marked `raspberrypi-edge`.

When no external model is available, the Application Layer may apply **deterministic or simulated** behavior for development and demonstration; the architecture still treats that path as “processing” logically because it produces structured inference output distinct from raw CRUD.

Key responsibilities:

- Accept image or feature input in a contract agreed with the application tier.
- Run detection, embedding, or classification pipelines as implemented (including post-ingest disease analysis).
- Return normalized results (labels, confidence, geometry) to the Application Layer.
- Enqueue scheduled edge captures without requiring an operator to be present.
- Remain stateless with respect to long-term domain truth; persistence is always via the application and data layers.

2.4 DATA LAYER DESCRIPTION

The Data Layer is the **authoritative foundation** for AgriHome. The relational store holds entities and relationships that satisfy SRS concepts: plant health records over time, capture references, environmental or monitoring events as modeled, mesh membership, schedules, and Summer 2026 **edge** tables

(edge_devices, edge_device_commands, pose sequences/poses). Object storage holds binary assets (camera frames, user uploads) referenced by identifiers or URLs in the relational model.

Optional components such as a **vector database** participate in this layer when similarity or embedding search is required; they remain secondary to the relational system of record.

Key responsibilities:

- Persist and retrieve structured domain and edge data with consistency expectations documented per operation.
- Store and stream large binaries efficiently.
- Enforce integrity through schema, constraints, and application-side validation.
- Support backup, migration, and operational monitoring consistent with deployment policy.

2.5 HARDWARE LAYER DESCRIPTION

The Hardware Layer is the **physical capture plane**. Raspberry Pi devices run Agri-Home/klipper with `fswebcam` and the `agrihome_agent`, which auto-provisions into AgriHome, reports online status, and delivers imagery by answering queued commands (primary) or by exposing an optional HTTP streamer that the Application Layer can pull when the Pi is LAN-reachable from the server.

Key responsibilities:

- Provide durable device identity and local agent configuration.
- Expose JPEG snapshot endpoints for server-side and agent-local capture.
- Execute pose-aware multi-angle capture when schedules or operators request it.
- Avoid autonomous plant-care actuation; motion fields may be stubbed pending hardware bring-up.

2.6 LAYERED ARCHITECTURE FIGURE

Figure 1 summarizes the five layers and primary external actors. Companion diagrams in Section 3 refine internal subsystems and data flow.

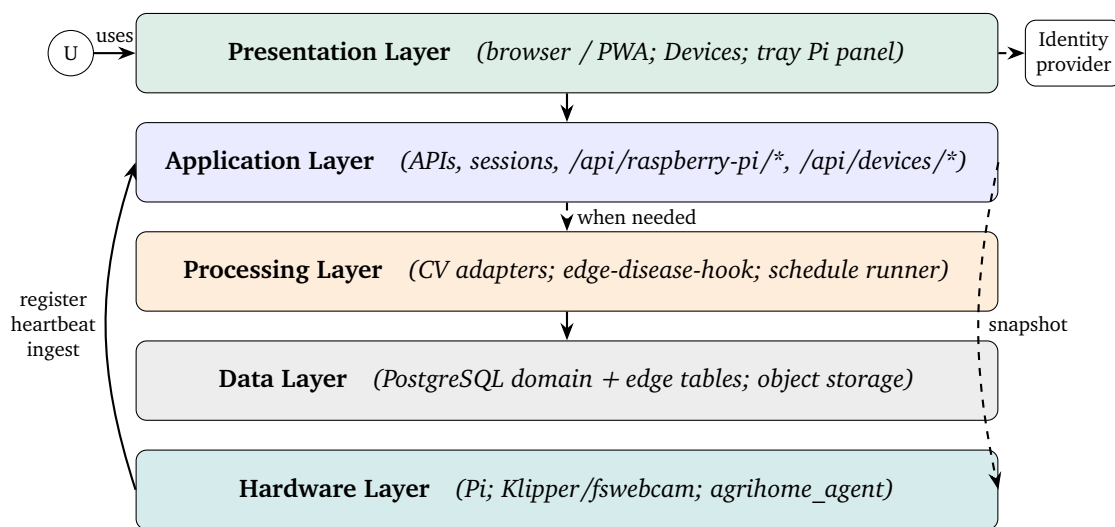
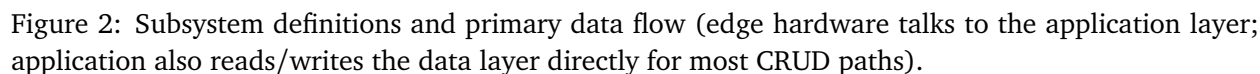


Figure 1: AgriHome Vision Console: five-layer architecture with Pi/Klipper edge trust boundaries.

2.7 SYSTEM WORKFLOW OVERVIEW

A typical end-to-end path begins when a signed-in user opens a **tray** or **Devices** view in the Presentation Layer. The client requests data from the Application Layer, which reads the **Data Layer** (relational and, when needed, media URLs). For **Take Picture**, the Application Layer primarily queues a command that `agrihome_agent` claims on heartbeat, captures via `fswebcam/save_image.sh`, and multipart-ingests; if an optional streamer URL is reachable it may pull a still first. When vision-assisted analysis is configured, the Processing Layer's edge disease hook may classify the capture; results return as structured data and are written to `prediction_results / reports`. Scheduled `raspberry-pi-edge` policies are advanced by the capture schedule runner. Updated state flows back to the Presentation Layer for display.

This section provides a consolidated **subsystem definition** and **data-flow** narrative for the AgriHome Vision Console, including the Summer 2026 Raspberry Pi / Klipper edge path. Figure 2 maps major building blocks to the five layers introduced in Section 2. The text following the figure walks through each layer and explains how information moves from user or agent intent to durable storage and back.



The Presentation Layer is organized around **workspaces** that match operator mental models. **Authentication and registration** establish identity and tie the browser to a server session. The **dashboard** summarizes tray health and recent activity. The **tray console** shows imagery, monitoring charts, and plant rows for a single tray, including a **Raspberry Pi panel** with Take Picture, device link, and Klipper/streamer URL edit. The **Devices** view lists provisioned edge devices (serial, status, last heartbeat, linked tray). **Plant detail** focuses on one organism. **Mesh views** aggregate trays. The **schedule editor** binds capture policy to trays or meshes, including raspberry-pi-edge destinations. The **PWA shell** supplies offline-friendly installation where supported.

3.2 APPLICATION LAYER COMPOSITION

page 15 of 40

ternal results to JSON or GraphQL shapes consumed by the UI or agent.

3.3 PROCESSING LAYER COMPOSITION

The Processing Layer exposes inference contracts and asynchronous capture helpers. A **species or leaf adapter** may classify a single-plant photo. A **tray vision adapter** may estimate plant count or regions. The **edge disease hook** optionally classifies captures after Pi ingest. The **capture schedule runner** evaluates due schedules with destination raspberry-pi-edge and enqueues `capture_now` (optionally with `runPoses`). **Result normalization** keeps bounding boxes, labels, and scores consistent before persistence.

3.4 DATA LAYER COMPOSITION

The Data Layer splits **transactional** concerns from **bulk media**. Write coordination sequences inserts so relational rows and file metadata stay aligned. Integrity constraints guard keys and ownership. Query execution runs against **PostgreSQL**, which holds trays, plants, meshes, schedules, monitoring events, predictions, reports, and edge tables (`edge_devices`, `edge_device_commands`, `capture_pose_sequences`, `capture_poses`, `capture_schedules`, `camera_captures`). The **object store** retains images; the relational model stores pointers.

3.5 HARDWARE LAYER COMPOSITION

The Hardware Layer is the bench capture plane: a **Raspberry Pi** host, **Agri-Home/klipper** with `fswebcam`, optional HTTP streamer, and `agrihome_agent`. Agents register and heartbeat; operators trigger Take Picture; schedules drive multi-pose walks. Primary capture is agent + `fswebcam`; optional server-side stills use a LAN-reachable `klipper_url`.

3.6 END-TO-END DATA FLOW SUMMARY

The data flow begins when a user interacts with the Presentation Layer or when an edge agent heartbeats. Requests pass to the Application Layer, which authorizes and validates them. Ordinary reads query the Data Layer. Take Picture primarily queues a command claimed by the Hardware Layer agent (`fswebcam`); it may optionally pull JPEG bytes from an HTTP streamer. Vision-assisted actions call the Processing Layer, then commit structured outputs through the Data Layer. Responses propagate back to the Presentation Layer (or agent) for rendering or further capture steps.

4 DATA LAYER SUBSYSTEMS

The Data Layer provides durable storage for all AgriHome domain state and media. It is structured into subsystems that mirror the reference senior-design pattern: coordinated writes, integrity checks, query execution, the primary relational database, and dedicated image storage.

4.1 WRITE COORDINATION SUBSYSTEM

The **write coordination** subsystem sequences operations that must stay consistent across multiple rows or across database rows and file metadata (for example, creating a plant and linking an uploaded photo). It receives structured write intents from the Application Layer and orders them so partial failures can be detected and reported.

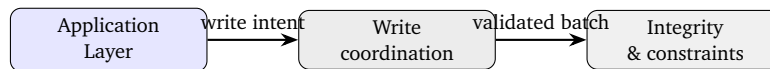


Figure 3: Write coordination subsystem: accepts write intents and forwards them toward integrity checks.

4.1.1 ASSUMPTIONS

- The Application Layer supplies writes in transactional units appropriate to each use case.
- Concurrent writers may exist; isolation expectations are defined per operation.
- Object storage availability may differ from database availability; coordination must surface errors clearly.

4.1.2 RESPONSIBILITIES

- Accept batched write intents from domain services.
- Order dependent operations (metadata before binary commit or the reverse, per policy).
- Expose success/failure to the Application Layer for user-visible error handling.

4.1.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-W1	Application → Write co-ordination	Domain write DTOs	Ordered write plan
D-W2	Write co-ordination → Integrity	Planned writes	Validation requests

Table 2: Write coordination subsystem interfaces

4.2 INTEGRITY AND CONSTRAINT ENFORCEMENT SUBSYSTEM

This subsystem enforces **schema rules, referential integrity, and business invariants** before data reaches the query executor. It corresponds to database constraints plus application-level checks that are not expressible as single-column rules.

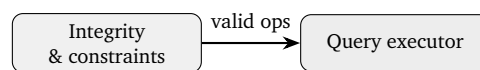


Figure 4: Integrity subsystem: validates operations before execution.

4.2.1 ASSUMPTIONS

- Invalid data may arrive from bugs or malicious clients; all paths must be validated server-side.
- Constraint definitions evolve with schema migrations.

4.2.2 RESPONSIBILITIES

- Reject writes that violate keys, foreign keys, or required fields.
- Apply cross-field rules (for example, ownership of a tray by the authenticated account).
- Return structured validation failures to the Application Layer.

4.2.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-I1	Write coordination → Integrity	Planned writes	Pass/fail + reasons
D-I2	Integrity → Query executor	Validated SQL ops	Executable statements

Table 3: Integrity and constraint enforcement subsystem interfaces

4.3 QUERY AND TRANSACTION EXECUTION SUBSYSTEM

The **query executor** runs SQL against PostgreSQL: transactional reads, inserts, updates, and deletes issued after validation. It returns rows, counts, or errors to the Application Layer.

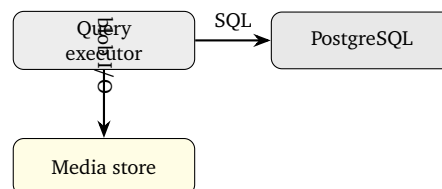


Figure 5: Query executor: issues SQL and coordinates binary I/O with storage.

4.3.1 ASSUMPTIONS

- Connection pooling is managed by the Application Layer infrastructure.
- Long-running analytics queries are out of scope for interactive paths.

4.3.2 RESPONSIBILITIES

- Execute parameterized statements for safety.
- Commit or roll back transactions per request scope.
- Surface deadlocks and transient errors for retry policy in higher layers.

4.3.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-Q1	Integrity → Query executor	Validated ops	Result sets / status
D-Q2	Query executor → PostgreSQL	SQL + params	Rows / ack
D-Q3	Query executor → Media store	Put/get requests	Handles / streams

Table 4: Query and transaction execution subsystem interfaces

4.4 POSTGRESQL DOMAIN DATABASE SUBSYSTEM

The **relational database** is the system of record for trays, plants, meshes, schedules, monitoring events, predictions, reports, media linkage, and Summer 2026 **edge** entities. Core edge-related tables include `edge_devices`, `edge_device_commands`, `capture_pose_sequences`, `capture_poses`, `capture_schedules`, `camera_captures`, `plants`, and `prediction_results`. It satisfies SRS needs for structured historical data, dashboard queries, and device command queues.



Figure 6: PostgreSQL domain database subsystem (includes edge device and pose tables).

4.4.1 ASSUMPTIONS

- Schema is versioned; migrations (including 009–011 edge migrations) are applied through controlled processes.
- Backup and restore procedures meet institutional RPO/RTO.

4.4.2 RESPONSIBILITIES

- Persist authoritative entities and relationships, including device registry and command queues.
- Support indexed access paths for tray/plant/mesh and device/heartbeat navigation.
- Retain audit-relevant history as required by product policy.

4.4.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-P1	Query executor → PostgreSQL	SQL	Tabular results
D-P2	PostgreSQL → Query executor	Internal state	Constraints / errors

Table 5: PostgreSQL domain database subsystem interfaces

4.5 EDGE DOMAIN PERSISTENCE SUBSYSTEM

Edge domain persistence groups the relational structures that back Raspberry Pi integration: device identity and Klipper/streamer URL `klipper_url` (`edge_devices`), pending/completed commands (`edge_device_commands`), multi-angle pose sequences (`capture_pose_sequences`, `capture_poses`), schedule rows targeting raspberry-pi-edge (`capture_schedules`), capture metadata including hinge/motor/pose fields (`camera_captures`), plant linkage (`plants`), and optional inference outcomes (`prediction_results`).

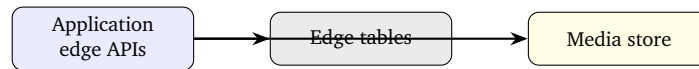


Figure 7: Edge domain persistence: relational edge tables plus capture binaries.

4.5.1 ASSUMPTIONS

- `tray_systems.edge_device_id` links at most one active device per tray policy as implemented.
- Command rows are claimed atomically on heartbeat to avoid double execution.

4.5.2 RESPONSIBILITIES

- Store hashed device credentials and heartbeat timestamps.
- Queue and complete capture commands with optional pose-run payloads.
- Retain pose sequences and capture metadata for multi-angle workflows.

4.5.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-E1	Application → Edge tables	Device/command/pose DTOs	Persisted rows
D-E2	Application → Media + captures	Image bytes + meta-data	Capture record + URL

Table 6: Edge domain persistence subsystem interfaces

4.6 IMAGE AND MEDIA STORAGE SUBSYSTEM

Image and media storage holds camera frames, user uploads, and other large binaries. The Application Layer stores objects and retains stable references in the relational database for use in URLs served back to the Presentation Layer.

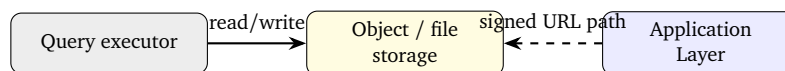


Figure 8: Image and media storage subsystem and relationship to application reads.

4.6.1 ASSUMPTIONS

- Storage roots are configurable per deployment environment.
- Virus scanning or content policy, if required, occurs at integration boundaries.

4.6.2 RESPONSIBILITIES

- Store and retrieve binaries keyed by application-managed paths or IDs.
- Enforce size and type limits cooperatively with validation in the Application Layer.

4.6.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
D-M1	Query executor → Media store	Bytes + path	Ack / handle
D-M2	Media store → Application	Stream requests	File bytes
D-M3	Application → Presentation	Media URL	Rendered imagery

Table 7: Image and media storage subsystem interfaces

5 PRESENTATION LAYER SUBSYSTEMS

The Presentation Layer implements all user-visible behavior described at a high level in the SRS: authentication, dashboards, plant health display, historical views, scheduling, mesh-oriented navigation, and Summer 2026 **edge device** controls (Devices list, tray Raspberry Pi panel, Take Picture). Each subsystem below corresponds to a coherent screen family or cross-cutting UI concern.

5.1 AUTHENTICATION AND SESSION UI SUBSYSTEM

This subsystem collects credentials or OAuth flows, exchanges tokens for server sessions, and guards entry to protected routes. It satisfies SRS **user authentication** expectations at the UX level.



Figure 9: Authentication UI: user actions become authenticated API calls.

5.1.1 ASSUMPTIONS

- Users have network connectivity for initial sign-in.
- Identity provider availability matches organizational SLA.

5.1.2 RESPONSIBILITIES

- Render sign-in, registration, and sign-out affordances.
- Hold short-lived client tokens only as required by the chosen identity integration.
- Redirect unauthenticated visitors away from protected console routes.

5.1.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-A1	User → Auth UI	Credentials / OAuth	Sign-in intent
P-A2	Auth UI → Application	Session exchange requests	HTTP-only session

Table 8: Authentication and session UI subsystem interfaces

5.2 DASHBOARD AND OVERVIEW SUBSYSTEM

The **dashboard** aggregates tray summaries, recent captures or predictions where available, and entry points into deeper views. It supports SRS **dashboard interface** and high-level plant status visibility.

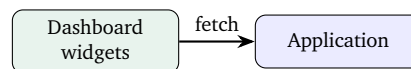


Figure 10: Dashboard subsystem requests summary aggregates from the application tier.

5.2.1 ASSUMPTIONS

- Summary data volume is bounded for interactive load times.
- Chart libraries render only on supported browsers.

5.2.2 RESPONSIBILITIES

- Display tray list and key health indicators.
- Link to tray detail, plants, and schedules.
- Surface empty and error states clearly.

5.2.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-D1	Dashboard → Application	Filter context	Summary DTOs
P-D2	Application → Dashboard	JSON payloads	Rendered charts / lists

Table 9: Dashboard and overview subsystem interfaces

5.3 TRAY MONITORING CONSOLE SUBSYSTEM

The **tray console** shows the active tray: latest imagery, monitoring trends, plant thumbnails, events, optional tray-level vision uploads, and the embedded **Raspberry Pi panel**. It aligns with **automatic image capture** presentation and tray-scoped monitoring in the SRS.



Figure 11: Tray console: drives reads and optional vision requests through the application layer.

5.3.1 ASSUMPTIONS

- Each tray identifier maps to authorized user scope.
- Large images may be progressively loaded or thumbnailed.
- A tray may be linked to zero or one `edge_devices` row.

5.3.2 RESPONSIBILITIES

- Render tray imagery, charts, and plant rows.
- Submit tray-scoped uploads or vision jobs as permitted.
- Host the Raspberry Pi panel affordances (link device, Take Picture, Klipper/streamer URL).
- Poll or refresh data according to UX policy.

5.3.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-T1	Tray UI → Application	Tray ID, actions	Tray detail DTO
P-T2	Tray UI → Application	Image multipart	Vision job result

Table 10: Tray monitoring console subsystem interfaces

5.4 PLANT DETAIL AND HEALTH HISTORY SUBSYSTEM

Plant detail focuses on a single plant: editable metadata, photo gallery, health reports over time, and scoped monitoring logs. It directly supports SRS **plant health status** and **historical data display**.

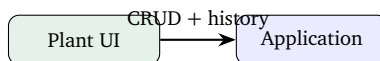


Figure 12: Plant detail subsystem: CRUD and reporting views.

5.4.1 ASSUMPTIONS

- Users may edit only plants they own or that policy allows.
- Report history may be lengthy; pagination or lazy loading applies.

5.4.2 RESPONSIBILITIES

- Display health timeline and latest findings.
- Support photo upload and deletion with confirmation.
- Invoke “add plant from photo” flows when enabled.

5.4.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-P1	Plant UI → Application	Plant ID	Detail + reports
P-P2	Plant UI → Application	Edits / uploads	Updated plant

Table 11: Plant detail and health history subsystem interfaces

5.5 MESH AND TOPOLOGY SUBSYSTEM

Mesh views let users group trays for aggregated charts and cross-tray plant listings, supporting experimental layouts described in the SRS’s organizational needs without mandating a particular hardware topology.

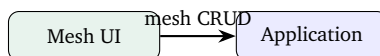


Figure 13: Mesh subsystem interacts with mesh and member tray aggregates.

5.5.1 ASSUMPTIONS

- Mesh membership changes are infrequent relative to reads.

5.5.2 RESPONSIBILITIES

- Create and list meshes; show merged monitoring where available.
- Navigate from mesh to constituent trays and plants.

5.5.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-M1	Mesh UI → Application	Mesh ID / payload	Mesh DTO

Table 12: Mesh and topology subsystem interfaces

5.6 SCHEDULE MANAGEMENT SUBSYSTEM

The **schedule editor** binds capture timing to trays or meshes, supporting SRS **automatic image capture** policies at the configuration level.

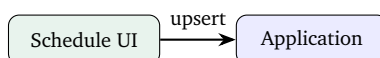


Figure 14: Schedule subsystem persists policies via application APIs.

5.6.1 ASSUMPTIONS

- Schedules are stored in a timezone-aware or UTC-normalized manner in the data layer.

5.6.2 RESPONSIBILITIES

- List and edit schedules with clear scope (tray vs. mesh).
- Support destination `raspberry-pi-edge` for edge capture policies.
- Validate user inputs (cadence, start/end) before submission.

5.6.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-S1	Schedule UI → Application	Schedule DTO	Persisted schedule

Table 13: Schedule management subsystem interfaces

5.7 DEVICES UI SUBSYSTEM

The **Devices** navigation lists provisioned Raspberry Pi edge devices for the signed-in owner: CPU serial, online/offline status, last heartbeat, and linked tray. It is the fleet-level companion to the per-tray Raspberry Pi panel.

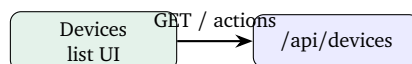


Figure 15: Devices UI: operator fleet view over edge device APIs.

5.7.1 ASSUMPTIONS

- Only devices owned by the authenticated account are visible.
- Heartbeat stale windows determine online presentation.

5.7.2 RESPONSIBILITIES

- Render device inventory and status.
- Navigate to linked trays when present.
- Surface revoke / rotate-key actions when exposed by product policy.

5.7.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-DV1	Devices UI → Application	Session context	Device list DTO
P-DV2	Devices UI → Application	Device action payload	Updated device state

Table 14: Devices UI subsystem interfaces

5.8 TRAY RASPBERRY PI PANEL AND TAKE PICTURE SUBSYSTEM

The **tray Raspberry Pi panel** embeds edge controls on tray detail: link/unlink device, edit and save **Klipper/streamer** URL (`klipper_url`), generate poses from plant layout, and **Take Picture**. Take Picture posts `action=capture` to `/api/devices/{id}`; the UI primarily expects a queued-command acknowledgment for agent `fswebcam` capture, or an immediate image URL when an optional HTTP streamer is reachable.



Figure 16: Tray Pi panel: Take Picture and Klipper URL management via devices API.

5.8.1 ASSUMPTIONS

- Offline empty states are shown when no device is linked or heartbeat is stale.
- Klipper/streamer URL updates validate http/https schemes before save.

5.8.2 RESPONSIBILITIES

- Trigger Take Picture and display returned capture or queue status.
- Persist Klipper URL edits (`updateKlipperUrl`; alias `updateMoonrakerUrl`).
- Support pose generation entry points for multi-angle capture.

5.8.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
P-PI1	Pi panel → Application	capture action	Image URL or queued command
P-PI2	Pi panel → Application	Klipper URL	Updated edge_devices row
P-PI3	Pi panel → Application	Pose generate request	Pose sequence DTO

Table 15: Tray Raspberry Pi panel and Take Picture subsystem interfaces

6 APPLICATION LAYER SUBSYSTEMS

The Application Layer implements the **control plane** of AgriHome: every client request is authenticated, validated, mapped to domain operations, and translated into data-layer and optional processing-layer calls. Device-key traffic from Raspberry Pi agents shares the same layer through dedicated `/api/raspberry-pi/*` routes. The subsystems below follow the decomposition style of the reference architectural document while reflecting AgriHome's agronomic and edge-capture domain.

6.1 API ENTRY AND ROUTING SUBSYSTEM

The **API entry** subsystem exposes HTTP routes and, where used, a GraphQL endpoint. It maps URLs and methods to internal handlers and enforces global concerns such as method allow-lists and body size limits.

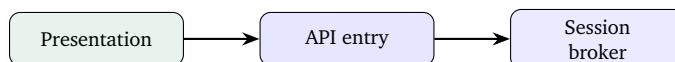


Figure 17: API entry: first server hop after the presentation layer.

6.1.1 ASSUMPTIONS

- All mutating operator operations require authenticated context unless explicitly public.
- Device routes authenticate with provisioning secrets or hashed device API keys rather than Firebase sessions.
- Rate limiting may be enforced externally (gateway) or internally per deployment.

6.1.2 RESPONSIBILITIES

- Accept REST and GraphQL traffic on documented paths, including `/api/raspberry-pi/*` and `/api/devices/*`.
- Dispatch to validation and domain orchestration pipelines.
- Normalize HTTP status codes and error envelopes.

6.1.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-R1	Presentation → API entry	HTTP requests	Routed handler
A-R2	API entry → Session broker	Raw request	Auth context

Table 16: API entry and routing subsystem interfaces

6.2 SESSION AND AUTHENTICATION BROKER SUBSYSTEM

This subsystem verifies **server sessions** established through the identity provider flow, binds the caller to an account key used for data scoping, and rejects stale or forged cookies.

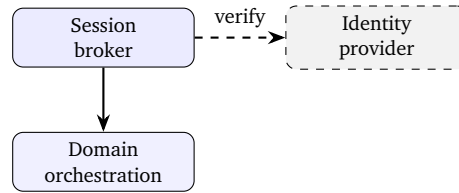


Figure 18: Session broker: establishes trusted account context for downstream work.

6.2.1 ASSUMPTIONS

- Session material is transported only over TLS.
- Clock skew between servers remains within acceptable bounds for token validation.

6.2.2 RESPONSIBILITIES

- Validate session cookies or bearer tokens per integration.
- Attach user identity and permissions context to the request scope.
- Support sign-out by invalidating server-side session state as implemented.

6.2.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-S1	API entry → Session broker	HTTP request	Principal or error
A-S2	Session broker → Domain	Account context	Scoped operations

Table 17: Session and authentication broker subsystem interfaces

6.3 INPUT VALIDATION SUBSYSTEM

Input validation enforces structural correctness: JSON schema, multipart boundaries, numeric ranges for schedules, and presence of foreign keys before any database call.

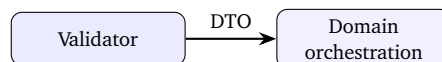


Figure 19: Validation subsystem: rejects bad input early.

6.3.1 ASSUMPTIONS

- All untrusted input arrives through the API entry path.

6.3.2 RESPONSIBILITIES

- Validate and sanitize payloads; coerce types where safe.
- Emit field-level errors consumable by the Presentation Layer.

6.3.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-V1	Session broker → Validator	Parsed body	Validated DTO

Table 18: Input validation subsystem interfaces

6.4 DOMAIN ORCHESTRATION SUBSYSTEM

Domain orchestration implements AgriHome use cases: tray and plant CRUD, mesh membership, schedule upserts, monitoring log writes, prediction snapshots, file handoffs, edge device lifecycle, and Take Picture coordination (queued agent fswecam capture or optional HTTP streamer still). It is the primary caller of the Data Layer and the orchestrator of Processing Layer calls.

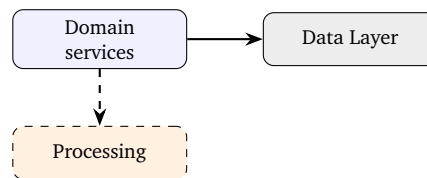


Figure 20: Domain orchestration: central business logic and integration point.

6.4.1 ASSUMPTIONS

- Domain rules may evolve faster than schema; feature flags may gate behavior.

6.4.2 RESPONSIBILITIES

- Implement transactional stories that satisfy SRS capabilities.
- Map database rows to API DTOs and GraphQL types.
- Coordinate retries or compensations when external services fail.

6.4.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-D1	Validator → Domain	Valid DTOs	Side effects / results
A-D2	Domain → Data Layer	SQL / storage ops	Persisted state
A-D3	Domain → Processing	Images / metadata	Inference DTOs

Table 19: Domain orchestration subsystem interfaces

6.5 GRAPHQL QUERY AND MUTATION FACADE SUBSYSTEM

Where enabled, **GraphQL** provides a typed alternative surface for reads and selected mutations (for example mesh creation, plant updates, schedule upserts). Resolvers delegate to the same domain orchestration as REST to avoid divergent business rules.

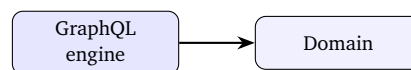


Figure 21: GraphQL facade: thin mapping to domain services.

6.5.1 ASSUMPTIONS

- GraphQL complexity limits protect against abusive queries.

6.5.2 RESPONSIBILITIES

- Parse, validate, and execute schema operations.
- Return typed errors suitable for client debugging.

6.5.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-G1	Presentation → GraphQL	Query/mutation	JSON result
A-G2	GraphQL → Domain	Resolver args	Domain outcomes

Table 20: GraphQL facade subsystem interfaces

6.6 RESPONSE AND ERROR HANDLING SUBSYSTEM

The **response handler** assembles payloads, sets cache headers where safe, logs structured errors, and ensures that sensitive internals are not leaked to clients—supporting SRS **security** themes at the API boundary.

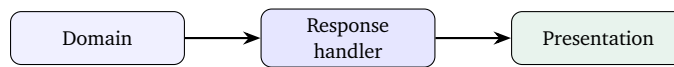


Figure 22: Response handler: maps internal results to HTTP/GraphQL responses.

6.6.1 ASSUMPTIONS

- Central logging infrastructure exists in deployment environments.

6.6.2 RESPONSIBILITIES

- Serialize success and error bodies consistently.
- Map domain exceptions to HTTP status codes.
- Redact secrets from logs and client messages.

6.6.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-E1	Domain → Response handler	Result / exception	Outbound DTO
A-E2	Response handler → Presentation	HTTP response	Rendered UX

Table 21: Response and error handling subsystem interfaces

6.7 RASPBERRY PI EDGE API SUBSYSTEM

The **Raspberry Pi edge API** (`/api/raspberry-pi/*`) is the device-facing control and ingest surface. Agents present a provisioning code at register time and thereafter a device API key (`X-Agrihome-Device-Key`). Routes cover register, heartbeat (claim pending commands), multipart ingest, command listing, active poses, and capture-plan (schedules + poses + limits).

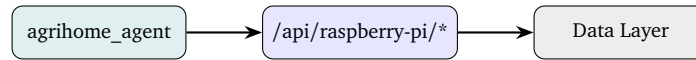


Figure 23: Raspberry Pi edge API: device-key routes into domain persistence.

6.7.1 ASSUMPTIONS

- API keys are stored only as hashes; plaintext is returned once at registration.
- Ingest may optionally trigger post-process hooks (plant attach, disease detection).

6.7.2 RESPONSIBILITIES

- Auto-provision `edge_devices` and linked trays on register.
- Update online status and claim `edge_device_commands` on heartbeat.
- Persist multipart frames to media storage and `camera_captures`.
- Serve pose sequences and capture plans for multi-angle runs.

6.7.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-RP1	Agent → register	Fingerprint + provisioning code	Device/tray IDs + API key
A-RP2	Agent → heartbeat	Device key	Pending commands
A-RP3	Agent → ingest	Multipart image + metadata	Capture DTO / URL
A-RP4	Agent → poses/plan	Device key	Pose sequence / schedule plan

Table 22: Raspberry Pi edge API subsystem interfaces

6.8 OPERATOR DEVICES API SUBSYSTEM

The **operator devices API** (`/api/devices` and `/api/devices/{id}`) is the session-authenticated control plane for Vision Console operators. It lists devices and performs actions such as capture (Take Picture), `linkTray`, `revoke`, `rotateKey`, and `updateKlipperUrl` (alias `updateMoonrakerUrl`). Capture primarily inserts a pending `capture_now` for the agent (`fswebcam`); when `klipper_url` is set it may attempt an optional HTTP streamer still first.

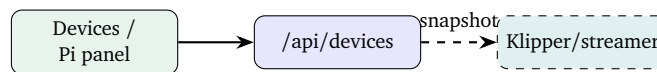


Figure 24: Operator devices API: Take Picture and device administration.

6.8.1 ASSUMPTIONS

- Operators may only act on devices they own.
- Optional streamer URL candidates include common nginx :80 webcam paths derived from `klipper_url`.

6.8.2 RESPONSIBILITIES

- Authorize and execute device actions from the Presentation Layer.
- Attempt server-side snapshot before queuing agent capture.
- Persist Klipper/streamer URL and linkage changes to `edge_devices` / trays.

6.8.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
A-DV1	Presentation → devices API	Session + filters	Device list
A-DV2	Presentation → devices API	capture action	Image URL or queued command
A-DV3	Presentation → devices API	Admin actions	Updated device/tray state

Table 23: Operator devices API subsystem interfaces

7 PROCESSING LAYER SUBSYSTEMS

The Processing Layer encapsulates **model-based and computer-vision** work for AgriHome, plus **asynchronous edge capture orchestration**. It is optional in the sense that core monitoring and CRUD remain available without external models; when present, it enriches the product with species hints, tray-level detections, post-ingest disease analysis, and scheduled Pi captures aligned with the SRS's emphasis on plant health insight and automatic image capture.

7.1 SPECIES AND LEAF INFERENCE ADAPTER SUBSYSTEM

This adapter accepts a **plant or leaf image** (or derived crop) and returns a structured classification: proposed species or cultivar labels, confidence scores, and auxiliary metadata for the Application Layer to store alongside user-confirmed fields.



Figure 25: Species inference adapter: hosted model runtime may be external.

7.1.1 ASSUMPTIONS

- Network latency between application and adapter is acceptable for interactive flows or runs asynchronously.
- Model versioning is tracked so historical reports can be interpreted correctly.

7.1.2 RESPONSIBILITIES

- Accept a well-defined inference request contract (image bytes or URL, optional tray/plant IDs).
- Run the trained classifier or delegate to a managed inference endpoint.
- Return normalized labels and scores without persisting domain state itself.

7.1.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
PR-S1	Application → Species adapter	Image + context	Label JSON
PR-S2	Species adapter → Application	HTTP response	Parsed DTO

Table 24: Species and leaf inference adapter interfaces

7.2 TRAY VISION AND LAYOUT ADAPTER SUBSYSTEM

The **tray vision** adapter analyzes a tray-level photograph to estimate plant count, regions of interest, or other layout features needed for monitoring dashboards. Outputs are merged into tray records or ephemeral UI state per product rules.

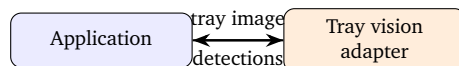


Figure 26: Tray vision adapter: round-trip inference for tray imagery.

7.2.1 ASSUMPTIONS

- Images meet minimum resolution and lighting heuristics expected by the model.
- A simulator or stub may substitute in development environments.

7.2.2 RESPONSIBILITIES

- Detect plants or regions; return geometry in a coordinate system agreed with the UI.
- Report confidence and failure modes explicitly.

7.2.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
PR-T1	Application → Tray adapter	Tray image	Detection set

Table 25: Tray vision and layout adapter interfaces

7.3 INFERENCE NORMALIZATION SUBSYSTEM

Normalization converts heterogeneous model outputs into a single internal schema (units, label vocabularies, bounding-box ordering) so the Application Layer can persist results without adapter-specific branches scattered through domain code.



Figure 27: Normalization: single canonical shape for downstream persistence.

7.3.1 ASSUMPTIONS

- Each adapter documents its native schema version.

7.3.2 RESPONSIBILITIES

- Map labels to controlled vocabularies where required.
- Clamp or filter low-confidence results per policy.
- Provide stable fields for database columns and UI rendering.

7.3.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
PR-N1	Adapter → Normalizer	Vendor JSON	Canonical inference DTO
PR-N2	Normalizer → Application	Canonical DTO	Ready for persist

Table 26: Inference normalization subsystem interfaces

7.4 EDGE DISEASE HOOK SUBSYSTEM

The **edge disease hook** (`edge-disease-hook`) runs after a successful Pi ingest or direct capture when a plant ID is present and species inference is configured. It invokes the leaf/species adapter on the stored frame, writes prediction/report outcomes through domain services, and records monitoring events on skip or failure—without failing the ingest itself.

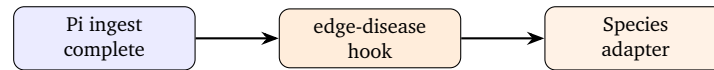


Figure 28: Edge disease hook: optional classification after Pi capture.

7.4.1 ASSUMPTIONS

- `CV_SPECIES_INFERENCE_URL` (or equivalent) may be unset; then the hook records a skip event.
- Hook execution is fire-and-forget relative to the HTTP ingest response path when configured asynchronously.

7.4.2 RESPONSIBILITIES

- Load capture bytes and call plant analysis services.
- Persist prediction/report results when inference succeeds.
- Emit monitoring events for skip, success, and failure cases.

7.4.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
PR-E1	Ingest path → Edge disease hook	Capture + plant context	Monitoring / prediction side effects
PR-E2	Edge disease hook → Species adapter	Image bytes	Diagnosis DTO

Table 27: Edge disease hook subsystem interfaces

7.5 CAPTURE SCHEDULE RUNNER SUBSYSTEM

The **capture schedule runner** (`capture:schedule-runner`) is a periodic job that evaluates due `capture_schedules`. For destination `raspberry-pi-edge`, it enqueues `capture_now` commands (optionally with `runPoses: true`) onto linked devices so `agrihome_agent` performs multi-angle capture without an operator present.

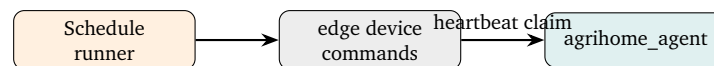


Figure 29: Capture schedule runner: enqueues edge capture work for agents.

7.5.1 ASSUMPTIONS

- The runner is invoked on a cron-like cadence (for example every minute).
- Schedules and device linkage are authoritative in the Data Layer.

7.5.2 RESPONSIBILITIES

- Select due schedules with edge destinations.
- Insert pending capture commands with pose-run flags as configured.
- Avoid blocking the interactive Application Layer request path.

7.5.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
PR-R1	Runner → Data Layer	Due schedule query	Schedule/device rows
PR-R2	Runner → Data Layer	capture_now in-tents	Pending commands

Table 28: Capture schedule runner subsystem interfaces

8 HARDWARE LAYER SUBSYSTEMS

The Hardware Layer comprises **bench capture endpoints**: Raspberry Pi devices running Agri-Home/klipper with fswbcam camera macros and the AgriHome edge agent. It is the physical source of tray imagery for automatic and operator-triggered capture. Unlike the Presentation Layer, these nodes authenticate with hashed **device API keys**, not Firebase user sessions.

8.1 RASPBERRY PI EDGE HOST SUBSYSTEM

Each provisioned **Raspberry Pi** (or compatible board) is a durable edge host with a CPU serial fingerprint used at registration. AgriHome stores the device in `edge_devices`, links it to a tray, and issues a one-time API key that the agent persists locally. Register uses `POST /api/raspberry-pi/register` with `DEVICE_PROVISIONING_SECRET`; the default model string is `raspberry-pi-klipper`.

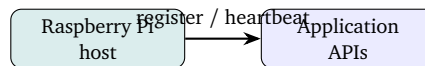


Figure 30: Pi edge host: register and heartbeat against AgriHome.

8.1.1 ASSUMPTIONS

- The host has network reachability to the AgriHome base URL.
- A shared provisioning secret is configured on both server and agent.

8.1.2 RESPONSIBILITIES

- Present stable hardware identity (CPU serial, MAC, hostname, model).
- Store the device API key securely on disk after first registration.
- Remain online via periodic heartbeats so Vision Console status stays current.

8.1.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
H-P1	Pi host → Application	Fingerprint + provisioning code	deviceId, trayId, API key
H-P2	Pi host → Application	Device-key heartbeat	Online status; pending commands

Table 29: Raspberry Pi edge host subsystem interfaces

8.2 KLIPPER / FSWEBCAM CAPTURE SUBSYSTEM

Agri-Home/klipper[6] provides G-code/macros for axis motion and `camera-macros/save_image.sh`, which captures JPEG stills with `fswbcam`. This is the **primary** Take Picture path when the agent claims `capture_now`. An optional LAN HTTP streamer base may be stored as `klipper_url` (migration `012_klipper_url` renamed `moonraker_url`→`klipper_url`) so the Application Layer can attempt a server-side still when reachable. Register/heartbeat accept deprecated `moonrakerUrl` as an alias for `klipperUrl`.

8.3.2 RESPONSIBILITIES

- Execute register / run loops against documented `/api/raspberry-pi/*` routes.
- Claim and complete edge commands; upload multipart captures with pose metadata.
- Fetch active pose sequences and capture plans for scheduled multi-angle work.

8.3.3 SUBSYSTEM INTERFACES

ID	Description	Inputs	Outputs
H-A1	Agent → Application	Device-key HTTP	Commands, plans, ingest ack
H-A2	Agent → Klipper/fswebcam	Snapshot / motion cues	Frame bytes; pose dwell

Table 31: agrihome_agent bridge subsystem interfaces

REFERENCES

- [1] Agrihome. *System Requirements Specification* (CSE 4316, Spring 2026). Companion requirements document.
- [2] Vercel, Inc. “Next.js Documentation.” <https://nextjs.org/docs> (accessed 2026).
- [3] PostgreSQL Global Development Group. “PostgreSQL Documentation.” <https://www.postgresql.org/docs/> (accessed 2026).
- [4] Google. “Firebase Authentication.” <https://firebase.google.com/docs/auth> (accessed 2026).
- [5] W3C. “Progressive Web Apps.” <https://www.w3.org/TR/appmanifest/> (accessed 2026).
- [6] Agri-Home. “klipper” companion firmware (camera-macros / fswebcam). <https://github.com/Agri-Home/klipper> (accessed 2026).
- [7] Arksine et al. “Moonraker: API Web Server for Klipper.” <https://moonraker.readthedocs.io/> (accessed 2026). Historical companion; AgriHome agent package still lives under Agri-Home/moonraker.
- [8] Agrihome. *Raspberry Pi + Klipper* operations guide (docs/ops/RASPBERRY_PI_KLIPPER.md), 2026. (Moonraker guide superseded stub.)